Computer Networks – Lab Report

Lab 4: Basic Networking Commands in Linux OS

Student Name:	Kishor Neupane
College:	New Summit College, Shantinagar, Kathmandu
Program:	BSc. CSIT – 4th Semester
Subject:	Computer Network
Lab Date:	June 9th, 2026
Submission Date:	June 16th, 2026
Instructor:	Shishir Ghimire

Objective

To understand the basics of networking commands available in the Linux operating system, including their description, purpose, syntax, and practical output.

Theory

Networking commands are built-in or installable utilities that allow users and administrators to diagnose, configure, and monitor network connectivity. In Linux, these commands are available through the terminal and are essential for troubleshooting network issues, viewing configurations, scanning ports, and transferring data. The following sections describe each command covered in Lab 4.

1. ping

Description

The **ping** command is one of the most fundamental network diagnostic tools. It sends ICMP (Internet Control Message Protocol) Echo Request packets to a target host and waits for ICMP Echo Reply packets. The round-trip time (RTT) measured for each packet indicates the latency between the source and destination. If no reply is received, it means the host is unreachable or is blocking ICMP traffic.

Use

Used to verify that a network host is reachable, measure network latency, check for packet loss, and confirm that TCP/IP networking is functioning correctly on the local machine.

Syntax

```
ping [options]
```

Examples

```
# Examples
ping google.com
ping 192.168.1.1
ping -c 4 google.com # Send only 4 packets
```

Figure / Sample Output

```
$ ping -c 4 google.com
PING google.com (142.250.195.46) 56(84) bytes of data.
64 bytes from 142.250.195.46: icmp_seq=1 ttl=115 time=12.3 ms
64 bytes from 142.250.195.46: icmp_seq=2 ttl=115 time=11.9 ms
64 bytes from 142.250.195.46: icmp_seq=3 ttl=115 time=12.1 ms
64 bytes from 142.250.195.46: icmp_seq=4 ttl=115 time=12.5 ms
--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max/mdev = 11.9/12.2/12.5/0.2 ms
```

Figure 1: Output of 'ping -c 4 google.com' showing round-trip time and packet statistics

2. hostname

Description

The **hostname** command displays or sets the system's hostname — the human-readable name that identifies the computer on a network. Every machine on a network should have a unique hostname. In Linux, the hostname is stored in the file /etc/hostname and is resolved using /etc/hosts or a DNS server.

Use

Used to quickly check the name of the current machine, which is important when managing multiple servers, writing shell scripts, or configuring network services.

Syntax

```
hostname [options]
```

Examples

```
# Examples
hostname # Display current hostname
hostname -I # Display all IP addresses of the host
hostname -f # Display the fully qualified domain name (FQDN)
```

Figure / Sample Output

```
$ hostname
kishor-pc

$ hostname -I
192.168.1.105 172.17.0.1

$ hostname -f
kishor-pc.local
```

Figure 2: Output of 'hostname' command showing system name and IP addresses

3. ifconfig / ip (Linux equivalent of ipconfig)

Description

In Linux, **ifconfig** (interface configuration) and the modern **ip** command are used in place of the Windows *ipconfig* command. They display network interface details such as IP address, subnet mask, MAC address, and packet statistics. **ifconfig** comes from the *net-tools* package while **ip** is from the newer *iproute2* package and is the recommended tool in modern Linux distributions.

Use

Used to view or configure network interfaces — checking assigned IP addresses, subnet masks, broadcast addresses, and MAC (hardware) addresses of all active interfaces.

Syntax

```
ifconfig [interface] OR ip addr show [interface]
```

Examples

```
# Examples
ifconfig # Show all active interfaces
ifconfig eth0 # Show only eth0 interface
ip addr show # Modern equivalent
ip addr show eth0 # Show specific interface
```

Figure / Sample Output

```
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.1.105 netmask 255.255.255.0 broadcast 192.168.1.255
inet6 fe80::a00:27ff:fe4e:66a1 prefixlen 64 scopeid 0x20<link>
ether 08:00:27:4e:66:a1 txqueuelen 1000 (Ethernet)
RX packets 1245 bytes 1020480 (996.6 KiB)
TX packets 892 bytes 112340 (109.7 KiB)

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
```

Figure 3: Output of 'ifconfig' showing IP, MAC address, and traffic stats for network interfaces

4. traceroute

Description

The **traceroute** command (Windows: tracert) maps the path that network packets take from the source machine to a destination host. It works by sending packets with gradually increasing TTL (Time To Live) values. When a router decrements the TTL to zero, it sends back an ICMP 'Time Exceeded' message, revealing its IP and response time. This continues hop-by-hop until the destination is reached.

Use

Used to identify where packets are being delayed or dropped in the network path, diagnose routing issues, and understand the network topology between source and destination.

Syntax

```
traceroute [options]
```

Examples

```
# Examples
traceroute google.com
traceroute -n google.com # Skip DNS resolution (faster)
traceroute -m 20 google.com # Set max hops to 20
```

Figure / Sample Output

```
$ traceroute google.com
traceroute to google.com (142.250.195.46), 30 hops max, 60 byte packets
1 192.168.1.1 (192.168.1.1) 1.234 ms 1.089 ms 1.002 ms
2 10.0.0.1 (10.0.0.1) 5.612 ms 5.498 ms 5.321 ms
3 103.11.0.1 (103.11.0.1) 8.123 ms 8.045 ms 7.981 ms
4 72.14.215.165 (72.14.215.165) 11.234 ms 11.102 ms 10.984 ms
5 142.250.195.46 (142.250.195.46) 12.456 ms 12.311 ms 12.198 ms
```

Figure 4: Output of 'traceroute google.com' showing each hop and round-trip times

5. netstat

Description

The **netstat** (network statistics) command displays active network connections, listening ports, routing tables, and interface statistics. It is a versatile tool for monitoring what connections exist on a system and which ports are open. In modern Linux, it is being replaced by the **ss** command, but netstat remains widely used.

Use

Used by system administrators to monitor open/listening ports, detect unauthorized connections, view routing tables, and diagnose network service issues.

Syntax

```
netstat [options]
```

Examples

```
# Examples
netstat -tulnp # Show all listening TCP/UDP ports with PID
netstat -an # Show all connections numerically
netstat -r # Show kernel routing table
```

Figure / Sample Output

```
$ netstat -tulnp
Proto Recv-Q Send-Q Local Address Foreign Address State PID/Program
tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN 1023/sshd
tcp 0 0 127.0.0.1:3306 0.0.0.0:* LISTEN 1456/mysqld
tcp6 0 0 :::80 :::* LISTEN 1502/apache2
udp 0 0 0.0.0.0:68 0.0.0.0:* 987/dhclient
```

Figure 5: Output of 'netstat -tulnp' showing listening services and their ports

6. nslookup

Description

The **nslookup** (Name Server Lookup) command queries the DNS (Domain Name System) to find the IP address associated with a domain name or vice versa (reverse lookup). It can be run in interactive or non-interactive mode. It is useful for testing DNS resolution and diagnosing DNS configuration problems.

Use

Used to resolve domain names to IP addresses, perform reverse DNS lookups, query specific DNS record types (A, MX, NS, CNAME), and test DNS server responses.

Syntax

```
nslookup [domain] [DNS server]
```

Examples

```
# Examples
nslookup google.com # Basic DNS lookup
nslookup 8.8.8.8 # Reverse lookup of an IP
nslookup google.com 1.1.1.1 # Use Cloudflare DNS server
```

Figure / Sample Output

```
$ nslookup google.com
Server: 127.0.0.53
Address: 127.0.0.53#53

Non-authoritative answer:
Name: google.com
Address: 142.250.195.46
Name: google.com
Address: 2404:6800:4009:820::200e
```

Figure 6: Output of 'nslookup google.com' showing resolved IPv4 and IPv6 addresses

7. route (ip route show)

Description

The **route** command (or modern **ip route show**) displays and manipulates the kernel IP routing table. The routing table determines how packets are forwarded from one network to another. Each entry in the routing table specifies a destination network, gateway, netmask, and the interface to use. The default gateway (0.0.0.0) is the router that handles all traffic not destined for local subnets.

Use

Used to view the current routing table, add or delete static routes, and troubleshoot routing problems such as packets not reaching their destination.

Syntax

```
route -n OR ip route show
```

Examples

```
# Examples
route -n # Show routing table numerically
ip route show # Modern equivalent
ip route add 10.0.0.0/8 via 192.168.1.1 # Add a static route
```

Figure / Sample Output

```
$ route -n
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
0.0.0.0 192.168.1.1 0.0.0.0 UG 100 0 0 eth0
192.168.1.0 0.0.0.0 255.255.255.0 U 100 0 0 eth0
172.17.0.0 0.0.0.0 255.255.0.0 U 0 0 0 docker0
```

Figure 7: Output of 'route -n' showing kernel routing table with gateway and interface info

8. nmap

Description

The **nmap** (Network Mapper) command is a powerful open-source tool for network exploration and security auditing. It can discover hosts on a network, detect open ports and running services, identify the operating system of remote hosts, and detect firewall rules. nmap must be installed separately on Linux using *sudo apt install nmap*.

Use

Used by network administrators and security professionals to inventory the network, identify open ports and running services, detect vulnerabilities, and audit firewall rules.

Syntax

```
nmap [options]
```

Examples

```
# Examples
nmap 192.168.1.1 # Scan a single host
nmap -sV 192.168.1.1 # Detect service versions
nmap -p 80,443 192.168.1.0/24 # Scan specific ports on a subnet
nmap -O 192.168.1.1 # OS detection
```

Figure / Sample Output

```
$ nmap 192.168.1.1
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.1.1
Host is up (0.0030s latency).
Not shown: 995 closed tcp ports
PORT STATE SERVICE
22/tcp open ssh
80/tcp open http
443/tcp open https
Nmap done: 1 IP address (1 host up) scanned in 1.25 seconds
```

Figure 8: Output of 'nmap 192.168.1.1' showing open ports and services on target host

9. netcat (nc)

Description

The **netcat** (nc) command is a versatile networking utility often called the 'Swiss Army knife' of networking. It can create raw TCP or UDP connections, act as a simple server or client, transfer files, and scan for open ports. It reads and writes data across network connections directly from the command line, making it useful for debugging and scripting.

Use

Used for port scanning, transferring files over TCP/UDP, testing server connectivity, creating simple chat sessions, and as a backend tool in shell scripting pipelines.

Syntax

```
nc [options]
```

Examples

```
# Examples
nc -zv 192.168.1.1 22 # Check if port 22 is open
nc -zv google.com 80 # Test HTTP port on google.com
nc -l 1234 # Start a listener on port 1234
nc 192.168.1.2 1234 # Connect to listener
```

Figure / Sample Output

```
$ nc -zv 192.168.1.1 22
Connection to 192.168.1.1 22 port [tcp/ssh] succeeded!

$ nc -zv google.com 80
Connection to google.com 80 port [tcp/http] succeeded!

$ nc -zv 192.168.1.1 8080
nc: connect to 192.168.1.1 port 8080 (tcp) failed: Connection refused
```

Figure 9: Output of 'nc -zv' showing port connectivity test results

10. curl

Description

The **curl** (Client URL) command is a tool to transfer data to or from a server using various protocols including HTTP, HTTPS, FTP, SMTP, and more. It supports features such as cookies, authentication, proxy tunneling, and SSL certificates. curl is widely used in scripting, API testing, and automation tasks.

Use

Used to test web servers and REST APIs, download files, send POST/GET requests with custom headers or data, and check HTTP response codes from the command line.

Syntax

```
curl [options]
```

Examples

```
# Examples
curl https://google.com # Fetch webpage content
curl -I https://google.com # Fetch only HTTP headers
curl -o file.html https://example.com # Save output to file
curl -X POST -d 'key=val' https://api.example.com/endpoint
```

Figure / Sample Output

```
$ curl -I https://google.com
HTTP/2 301
location: https://www.google.com/
content-type: text/html; charset=UTF-8
date: Mon, 09 Jun 2026 10:32:45 GMT
server: gws
content-length: 220
x-xss-protection: 0
x-frame-options: SAMEORIGIN
```

Figure 10: Output of 'curl -I https://google.com' showing HTTP response headers

11. uname / lsb_release (Linux equivalent of systeminfo)

Description

In Linux, the **uname** and **lsb_release** commands serve the purpose of the Windows *systeminfo* command by displaying detailed information about the operating system and hardware. **uname -a** shows the kernel name, hostname, kernel release, version, machine architecture, and OS. **lsb_release -a** shows the Linux distribution name, release version, and codename.

Use

Used to identify the Linux distribution and version, kernel version, CPU architecture, and other system information — useful for compatibility checks and system administration.

Syntax

```
uname [options] OR lsb_release [options]
```

Examples

```
# Examples
uname -a # All system information
uname -r # Kernel release version only
lsb_release -a # Linux distribution details
```

Figure / Sample Output

```
$ uname -a
Linux kishor-pc 5.15.0-76-generic #83-Ubuntu SMP Thu Jun 15 19:16:32 UTC 2023
x86_64 x86_64 x86_64 GNU/Linux

$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description: Ubuntu 22.04.2 LTS
Release: 22.04
Codename: jammy
```

Figure 11: Output of 'uname -a' and 'lsb_release -a' showing OS and kernel details

Conclusion

In this lab, we studied eleven fundamental networking commands available in the Linux operating system. Each command serves a distinct purpose in network diagnostics, configuration, and communication:

- **ping** – checks host reachability and measures round-trip latency using ICMP packets.
- **hostname** – displays the machine's name on the network, along with its IP addresses.
- **ifconfig / ip addr** – shows detailed network interface information including IP, MAC, and traffic statistics.
- **traceroute** – reveals the full network path (hop-by-hop) from source to destination.
- **netstat** – lists active connections, open ports, and routing information on the system.
- **nslookup** – performs DNS lookups to resolve domain names to IP addresses and vice versa.
- **route / ip route** – displays the kernel routing table showing how packets are directed to their destinations.
- **nmap** – scans networks and hosts to discover open ports, running services, and OS information.
- **netcat (nc)** – creates raw TCP/UDP connections for port testing, file transfer, and network debugging.
- **curl** – transfers data to and from web servers using HTTP/HTTPS and other protocols.
- **uname / lsb_release** – provides system and OS information equivalent to Windows' systeminfo command.

These commands form the foundation of Linux network administration and are essential for any computer networking professional. Understanding their syntax, options, and output enables efficient network troubleshooting and management in real-world environments.

References

1. CN Lab Reference Material – https://drive.google.com/file/d/1iw30b3PgVY7CDEgzFfV2SISRrih7i4a5/view
2. CN Lab Reference Material 2 – https://drive.google.com/file/d/1RtryMWjRydXCgyePg7rFP657s3fWp7N0/view
3. Linux man pages – https://man7.org/linux/man-pages/
4. GNU/Linux Networking Commands – https://linuxhint.com/networking-commands-linux/